CSE 213 — Software Engineering

Refactoring and Code Smells

Lesson notes · Exam approach · TF practice with solutions · Quick notes (IJ Mam's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — refactoring questions — every recent paper
Lesson 18 — What Refactoring Is; Why, When and Its Problems — pp. 1–14
Lesson 19 — Code Smells — pp. 15–29
Lesson 20 — The Refactoring Catalogue and Worked TF Refactorings — pp. 30–47
Quick Notes — Refactoring — smell → refactoring map

NOTE Slide numbers refer to the PDF page numbers of the deck named under each lesson title. TF page numbers refer to 'CSE 213 All Past Papers.pdf' (70 pages); the newest paper (22 batch, labelled 2024-25 here) is a separate file, cited as '22 batch p.N'. Old course code: CSE 307. Page numbers refer to '7 - Refactoring.pdf' (48 pages). Smells not shown on the slides (refused bequest, parallel inheritance hierarchies) follow Fowler and refactoring.guru (marked 'textbook'). All refactored Java was compiled and run (sources in _build/java/l5).

Exam Map — What the TF Papers Ask

Session (paper)	Questions in this note file
2024-25 (CSE 213)	Q4(a) Student / Scholarship code: identify five bad smells and refactor (20) [L20] · Q4(b) Code smell vs technical debt vs refactoring; proactive vs reactive risk management (15) [L18]
2023-24 (CSE 213)	Q8(c) DiscountCalculator / OrderProcessor: bad smells + refactor (20) [L20]
2022-23 (CSE 307)	Q5(a) Define with examples: shotgun surgery, parallel inheritance hierarchies, refused bequest, feature envy (15) [L19] · Q5(b) Refactor: (i) renderBanner complex condition (ii) getSalary switch (iii) getPayAmount nested if (20) [L20]
2021-22 (CSE 307)	Q1(b) GetSuperHeroItem if-else chain → readable, O(1) amortised (13) [L20] · Q6(c) Define code smell; three examples (10) [L19]
2019-20 (CSE 307)	Q6(a) Code snippets showing Data Clump and Refused Bequest (20) [L19]
2017-18 (CSE 307)	Q2(c) Why refactoring is important in agile; examples (7) [L18]

EXAM FOCUS ★★★ every year: identify smells and refactor given code — 2024-25 (20 marks), 2023-24 (20), 2022-23 (15 + 20), 2021-22 (13 + 10), 2019-20 (20). Learn the smell → refactoring map (Quick Notes) and practise naming the smell, then the refactoring, then writing the after-code.

Lesson 18 — What Refactoring Is; Why, When and Its Problems

Pages 1–14 (7 - Refactoring)

MEMORIZE **Refactoring** (Fowler, noun): a change made to the internal structure of software to make it easier to understand and cheaper to modify without changing its observable behavior. (Verb): to restructure software by applying a series of refactorings without changing its observable behavior (p. 8).

- External behaviour unchanged (functional requirements kept), internal structure improved — 'improving the design after it has been written' (p. 2).
- Simple examples: consolidate duplicate conditional fragments (send() moved after the if/else, p. 3); replace magic number 9.81 with GRAVITATIONAL_CONSTANT (p. 4).
- Dangerous: can introduce bugs; managers see time not spent on features → make it systematic, incremental and safe (p. 5).
- Useful: designs are rarely right first time and requirements change; code often shrinks and becomes simpler (p. 6).
- Contrast with performance optimisation: also keeps functionality but often makes code harder to understand (p. 9).
- Safe refactoring: use named refactoring patterns, and **test constantly** — run the tests after every step (p. 10).

Why refactor (p. 11)	When (p. 12)	Problems (p. 13)
improves design (stops decay); easier to understand; helps find bugs; helps program faster	Rule of Three (third duplication → refactor); when adding a function (before or after); when fixing a bug; during code review	databases are tightly coupled and hard to change; published interfaces cannot change

Finding where to refactor: look for **bad smells** (p. 14); tools such as Codegrip, Checkstyle, PMD, FindBugs and SonarQube detect them automatically.

NOTE Technical debt (2024-25 Q4(b), textbook): the future cost of rework caused by choosing a quick or easy solution now; code smells are its symptoms, refactoring is how it is repaid. Interest = the extra effort every change costs while the debt remains.

Practice — Lesson 18

(a) Actual — 2024-25 Q4(b). (b) Actual — 2017-18 Q2(c).

(a) Define code smell, tech-debt and code refactoring. Differentiate proactive and reactive risk-management strategies with examples. (15)

(b) Why is refactoring important in agile development? Examples. (7)

✓ Solution

(a) Code smell = a surface symptom in code that usually indicates a deeper design problem (e.g. a 200-line method, duplicated code). Technical debt = NOTE above. Refactoring = Fowler's definition. Reactive risk strategy: act only after a risk occurs (fire-fighting) — e.g. a key developer leaves and the team then scrambles to hire and learn the code. Proactive: identify, analyse and plan for risks before they happen (risk table, mitigation, monitoring, contingency) — e.g. pair programming and documentation so knowledge is shared before anyone leaves. Proactive is cheaper and is real risk management; reactive is crisis management.

(b) Agile builds the design incrementally with little up-front design, and requirements change every sprint; without refactoring the code decays into duplication and long methods, and velocity falls. Refactoring with automated tests (TDD) keeps the design simple (XP 'simple design', 'refactor mercilessly'). Examples: extract a duplicated validation into one method after the third user story needs it; replace a growing switch on payment type with polymorphic classes when a new payment method is added.

Lesson 19 — Code Smells

Pages 15–29 (7 - Refactoring)

Group (p. 15)	Smells
Bloaters	Long Method, Large Class, Primitive Obsession, Long Parameter List, Data Clumps
OO abusers	Alternative Classes with Different Interfaces, Refused Bequest, Switch Statements, Temporary Field
Change preventers	Divergent Change, Parallel Inheritance Hierarchies, Shotgun Surgery
Dispensables	Comments, Duplicate Code, Data Class, Dead Code, Lazy Class, Speculative Generality
Couplers	Feature Envy, Inappropriate Intimacy, Incomplete Library Class, Message Chains, Middle Man

Smell	Symptom (slides)	Refactoring
Duplicated code (p. 17)	same lookup + exception in CustomerNameChanger and CustomerAddressChanger	Extract Method / Pull Up Method
Long method (p. 18)	processOrder validates, totals, discounts, formats and prints	Extract Method per step
Large class	does too much, low cohesion	Extract Class
Long parameter list (p. 19)	processPayment(name, email, cardNumber, expiry, amount)	Introduce Parameter Object
Divergent change (p. 21)	Employee changes for personal info, report format, email, pay rules	Extract Class per reason
Shotgun surgery (p. 22)	minimum-balance rule and message copied in debit, transfer …	Move Method / Inline into one class
Feature envy (p. 24)	Invoice.calculateDiscount uses only Customer's data	Move Method to Customer
Data clumps (p. 23)	fields that always travel together (street, postOffice, district)	Extract Class / Parameter Object
Primitive obsession (p. 25)	parallel arrays std_ID, std_name, std_marks	Replace Data Value with Object (Student class)
Switch statements (p. 25)	the same switch on a type code in many places	Replace Conditional with Polymorphism
Lazy class (p. 26)	Logger that only calls println	Inline Class
Data class (pp. 27–28)	only fields + getters/setters; behaviour elsewhere	Move Method into it (Order.getTotalPrice)
Comments (p. 29)	comments used as deodorant for bad code	Extract Method with a good name
Refused bequest (textbook)	subclass ignores/throws in inherited methods	Replace Inheritance with Delegation; Push Down
Parallel inheritance (textbook)	each new subclass in one hierarchy needs one in another	Move methods so one hierarchy absorbs the other

Practice — Lesson 19

(a) Actual — 2022-23 Q5(a). (b) Actual — 2019-20 Q6(a). (c) Actual — 2021-22 Q6(c).

(a) Define with short code examples: shotgun surgery, parallel inheritance hierarchies, refused bequest, feature envy. (4+4+4+3)

(b) Two program snippets showing the smells Data Clump and Refused Bequest. (20)

(c) Define code smell; three examples. (10)

✓ Solution

(a)/(b) Each smell with before → after (compiled):

```
// 2019-20 Q6(a), 2022-23 Q5(a) — code smells with short examples and fixes (compile-checked)
import java.util.ArrayList;
import java.util.List;

// DATA CLUMP: the same three fields travel together  ->  Extract Class / Introduce Parameter Object
class MeetingBefore {
    void schedule(int startDay, int startMonth, int startYear, int endDay, int endMonth, int endYear)
{ }
}
class SimpleDate { final int d, m, y; SimpleDate(int d, int m, int y) { this.d = d; this.m = m;
this.y = y; } }
class MeetingAfter { void schedule(SimpleDate start, SimpleDate end) { } }

// REFUSED BEQUEST: subclass inherits methods it does not want  ->  Replace Inheritance with
Delegation / push down
class Bird { void fly() { System.out.println("flying"); } void eat() { } }
class PenguinBefore extends Bird {
    void fly() { throw new UnsupportedOperationException("penguins can't fly"); }   // refuses the
bequest
}
abstract class Animal { void eat() { } }
class FlyingBird extends Animal { void fly() { } }
class Penguin extends Animal { void swim() { } }                                  // after: no
unwanted method

// FEATURE ENVY: a method uses another class's data more than its own -> Move Method
class Customer2 { double totalSpent; int yearsAsMember;
    double loyaltyDiscount() {                                                     // after: moved
here
        if (totalSpent > 1000 && yearsAsMember > 2) return totalSpent * 0.15;
        if (totalSpent > 500) return totalSpent * 0.10;
        return 0;
    }
}
class Invoice2 { double discountFor(Customer2 c) { return c.loyaltyDiscount(); } }

// SHOTGUN SURGERY: one rule copied into many classes -> Move Method / Extract Class to one place
class AccountRules { static final double MIN_BALANCE = 500;
    static void checkMinimum(double balanceAfter) {
        if (balanceAfter < MIN_BALANCE) throw new IllegalStateException("Balance would fall below " +
MIN_BALANCE);
    }
}
class Debit    { void run(double bal, double amt) { AccountRules.checkMinimum(bal - amt); } }
class Transfer { void run(double bal, double amt) { AccountRules.checkMinimum(bal - amt); } }

// PARALLEL INHERITANCE HIERARCHIES: every new Employee subclass needs a new Printer subclass ->
merge via polymorphism
abstract class Employee2 { abstract String report(); }           // after: each employee reports
itself
class Engineer2 extends Employee2 { String report() { return "Engineer report"; } }
class Salesman2 extends Employee2 { String report() { return "Salesman report"; } }
// before: Engineer/EngineerPrinter, Salesman/SalesmanPrinter grew in parallel
```

```
public class SmellExamples {
    public static void main(String[] args) {
        List<Employee2> staff = new ArrayList<>(List.of(new Engineer2(), new Salesman2()));
        for (Employee2 e : staff) System.out.println(e.report());
        Customer2 c = new Customer2(); c.totalSpent = 1200; c.yearsAsMember = 3;
        System.out.println("discount " + new Invoice2().discountFor(c));
    }
}
```

(c) Definition (Lesson 18) + any three rows of the table with a two-line example each.

Lesson 20 — The Refactoring Catalogue and Worked TF Refactorings

Pages 30–47 (7 - Refactoring)

Refactoring (p. 30)	Do	Slide example
Extract Method (pp. 31–32)	turn a fragment into a method named after its purpose	print_invoice → calculate_total
Extract Class (p. 33)	one class doing two jobs → move fields/methods to a new class	Person → TelephoneNumber
Extract Variable (pp. 34–35)	name parts of a complex expression	basePrice, isBulkOrder, discountRate
Replace Temp with Query (pp. 36–38)	temp → private method usable elsewhere	basePrice → getBasePrice()
Move Method (pp. 39–40)	move to the class whose data it uses	Car.calculateFuelEfficiency → Engine
Replace Conditional with Polymorphism (pp. 41–42)	each branch → overriding method	Bird.getSpeed per subclass
Separate Query from Modifier (pp. 43–44)	split a method that returns and changes state	getTotalOutstanding + setReadyForSummaries
Introduce Parameter Object (pp. 45–46)	group parameters that travel together	DateRange(start, end, includes)
Also: Introduce Null Object, Encapsulate Collection, Replace Nested Conditional with Guard Clauses		

```
// Extract Variable (p. 35)
double calculatePrice(double quantity, double itemPrice) {
    double basePrice = quantity * itemPrice;
    boolean isBulkOrder = quantity > 100;
    double discountRate = isBulkOrder ? 0.9 : 0.95;
    return basePrice * discountRate;
}
```

Practice — Lesson 20

(a) Actual — 2022-23 Q5(b). (b) Actual — 2021-22 Q1(b). (c) Actual — 2023-24 Q8(c). (d) Actual — 2024-25 Q4(a).

(a) Which refactorings, and the refactored code: (i) renderBanner with if ((platform.toUpperCase().indexOf("MAC") > -1) && (browser.toUpperCase().indexOf("IE") > -1) && wasInitialized() && resize > 0) (ii) getSalary() with switch (type) Football / Cricket / Badminton (iii) getPayAmount() with nested if-else on isDead, isSeparated, isRetired. (6+7+7)

(b) GetSuperHeroItem.getItem(heroName) is an if-else chain (IronMan → suit, Thor → hammer, CaptainAmerica → shield, Hawkeye → arrow, else null). Rewrite it to be easier to read and maintain, with constant amortised run time. (13)

(c) Define bad smells and refactoring; identify the smells in DiscountCalculator.calculateDiscount(customerType, productType, price, customerName, customerId) (if-chain on REGULAR/PREMIUM × ELECTRONICS/GROCERY with 10/5/20/10 %) and OrderProcessor.processOrder(...) (prints customer data twice, if-chains on the same strings, compares discount == price × 0.1 / 0.15 / 0.25), and revise the code. (20)

(d) Identify five bad smells in Student (type code 1 = UG / 2 = PG, street/postOffice/district, yearWiseGpa, isFulltime for PG only, calculateFee and printDetails switching on type) and Scholarship (isRegular(Student), checkEligibility with deep nesting and duplicated CGPA loop) and refactor each. (20)

✓ Solution

(a) (i) **Decompose Conditional / Extract Variable** — name each part (isMacOs, isIE, wasResized). (ii) **Replace Conditional with Polymorphism** — Player subclasses override getSalary() (type code removed). (iii) **Replace Nested Conditional with Guard Clauses** — return early; no temp 'result'.

```
// 2022-23 Q5(b) – (i) Decompose Conditional / Extract Variable, (ii) Replace Conditional with
Polymorphism,
// (iii) Replace Nested Conditional with Guard Clauses
class Banner {
    String platform = "MacOS", browser = "IE 11"; boolean initialized = true; int resize = 1;
    boolean wasInitialized() { return initialized; }
    void renderBanner() {
        final boolean isMacOs = platform.toUpperCase().indexOf("MAC") > -1;
        final boolean isIE = browser.toUpperCase().indexOf("IE") > -1;
        final boolean wasResized = resize > 0;
        if (isMacOs && isIE && wasInitialized() && wasResized) {
            System.out.println("render banner for Mac + IE");    // do something
        }
    }
}
abstract class Player {                                        // (ii)
    double baseSalary() { return 50000; }
    abstract double getSalary();
}
class FootballPlayer extends Player {
    double getWeeklyIncrement() { return 1000; }
    double getSalary() { return baseSalary() + getWeeklyIncrement() * 4; }
}
class CricketPlayer extends Player {
    double getMonthlyIncrement() { return 3000; }
    double getSalary() { return baseSalary() + getMonthlyIncrement(); }
}
class BadmintonPlayer extends Player {
    double getSalary() { return baseSalary(); }
}
class Payroll {                                                // (iii)
    boolean isDead, isSeparated, isRetired;
    double deadAmount() { return 1; } double separatedAmount() { return 2; }
    double retiredAmount() { return 3; } double normalPayAmount() { return 4; }
    double getPayAmount() {
        if (isDead) return deadAmount();
        if (isSeparated) return separatedAmount();
        if (isRetired) return retiredAmount();
        return normalPayAmount();
    }
}
```

```
public class Refactor2223 {
    public static void main(String[] args) {
        new Banner().renderBanner();
        for (Player p : new Player[]{new FootballPlayer(), new CricketPlayer(), new
BadmintonPlayer()})
            System.out.println(p.getClass().getSimpleName() + " " + p.getSalary());
        Payroll pr = new Payroll(); pr.isRetired = true;
        System.out.println("pay " + pr.getPayAmount());
    }
}
```

(b) Smells: long if-else chain (switch smell), magic strings, public fields. Refactoring: **replace conditional with a lookup table** (HashMap) filled once — getItem is O(1) amortised, a new hero is one register() call, and keys are lower-cased to keep equalsIgnoreCase behaviour.

```
// 2021-22 Q1(b) — if-else chain on names -> lookup table (HashMap): readable, O(1) amortised
import java.util.HashMap;
import java.util.Map;

class SuperHeroItem {
    final String name; SuperHeroItem(String n) { name = n; }
    public String toString() { return name; }
}
class GetSuperHeroItem {
    private final Map<String, SuperHeroItem> itemByHero = new HashMap<>();
    GetSuperHeroItem() {
        register("IronMan", new SuperHeroItem("suit"));
        register("Thor", new SuperHeroItem("hammer"));
        register("CaptainAmerica", new SuperHeroItem("shield"));
        register("Hawkeye", new SuperHeroItem("arrow"));
    }
    void register(String hero, SuperHeroItem item) { itemByHero.put(hero.toLowerCase(), item); }  //
new hero = one call
    SuperHeroItem getItem(String heroName) {
        return heroName == null ? null : itemByHero.get(heroName.toLowerCase());   // case-
insensitive, O(1)
    }
}
public class SuperHeroItems {
    public static void main(String[] args) {
        GetSuperHeroItem g = new GetSuperHeroItem();
        System.out.println(g.getItem("thor") + " " + g.getItem("IRONMAN") + " " + g.getItem("Hulk"));
    }
}
```

(c) Smells: **long parameter list** and **data clump** (customerName, customerId, customerType); **unused parameters** (name and id in calculateDiscount); **switch statements** — if-chains on the type strings, repeated in both classes; **magic numbers/strings**; **duplicated code** (customer data printed twice); **long method** (processOrder); a logic bug hidden by the smells — discount is compared with price × 0.1 / 0.15 / 0.25 though the rates are 0.05, 0.1, 0.2, and doubles are compared with ==. Refactorings: Introduce Parameter Object (Customer, Product), Replace Type Code with enum + Replace Conditional with Polymorphism (rates inside CustomerType), Extract Method for printing, compute and print the rate instead of comparing doubles.

```
// 2023-24 Q8(c) — DiscountCalculator / OrderProcessor after refactoring
// Smells: long parameter list + data clump (customerName, customerId, customerType) -> Introduce
Parameter Object Customer;
// unused parameters (name, id in calculateDiscount); switch-like if-chain on type codes ->
polymorphic CustomerType enum;
// duplicated code (printing customer data twice, string comparisons repeated); magic numbers ->
constants in the enum;
// long method processOrder -> Extract Method; bug: discount == price*0.15 / 0.25 never match the
rates (0.05, 0.1, 0.2)
// and float comparison with == -> print the rate instead; feature envy on strings -> methods on
Product/Customer.
enum ProductType { ELECTRONICS, GROCERY, OTHER }
enum CustomerType {
    REGULAR(0.10, 0.05), PREMIUM(0.20, 0.10);
    private final double electronicsRate, groceryRate;
    CustomerType(double e, double g) { electronicsRate = e; groceryRate = g; }
    double rateFor(ProductType p) {
        switch (p) {
            case ELECTRONICS: return electronicsRate;
            case GROCERY:     return groceryRate;
            default:          return 0;
        }
    }
}
class Customer {
    final String name, id; final CustomerType type;
    Customer(String name, String id, CustomerType type) { this.name = name; this.id = id; this.type =
type; }
}
class Product {
    final ProductType type; final double price;
    Product(ProductType type, double price) { this.type = type; this.price = price; }
}
class DiscountCalculator {
    double discountRate(Customer c, Product p) { return c.type.rateFor(p.type); }
    double calculateDiscount(Customer c, Product p) { return p.price * discountRate(c, p); }
}
```

```
class OrderProcessor {
    private final DiscountCalculator calculator = new DiscountCalculator();
    void processOrder(Customer c, Product p) {
        double rate = calculator.discountRate(c, p);
        double discount = p.price * rate;
        printCustomer(c);
        printProduct(p);
        printPrices(p.price, discount, rate);
    }
    private void printCustomer(Customer c) {
        System.out.println("Processing order for " + c.name + " (" + c.id + "), " + c.type + "
customer");
    }
    private void printProduct(Product p) { System.out.println(p.type + " item processed"); }
    private void printPrices(double price, double discount, double rate) {
        System.out.println("Original price: " + price);
        System.out.println(rate == 0 ? "No discount applied" : Math.round(rate * 100) + "% discount
applied: " + discount);
        System.out.println("Final price: " + (price - discount));
    }
}
public class OrderRefactored {
    public static void main(String[] args) {
        OrderProcessor op = new OrderProcessor();
        op.processOrder(new Customer("Rina", "C-17", CustomerType.PREMIUM), new
Product(ProductType.ELECTRONICS, 1000));
        op.processOrder(new Customer("Karim", "C-18", CustomerType.REGULAR), new
Product(ProductType.GROCERY, 200));
        op.processOrder(new Customer("Karim", "C-18", CustomerType.REGULAR), new
Product(ProductType.OTHER, 50));
    }
}
```

(d) Five smells → refactorings: (1) **type code + switch** (type 1/2 in calculateFee and printDetails) → Replace Type Code with Subclasses (UndergraduateStudent, PostgraduateStudent) + polymorphism; (2) **data clump** street/postOffice/district → Extract Class Address; (3) **temporary field** isFulltime used only for PG → move into PostgraduateStudent; (4) **feature envy** — Scholarship computes regularity and CGPA from Student's data → Move Method (isRegular, cgpa into Student); (5) **duplicated code** — CGPA loop in Student and Scholarship → Extract Method cgpa(); plus **long method / deep nesting** in checkEligibility → guard clauses, and magic numbers 50000/40000/48000/3.5 → named constants. ('stuent.type' typo also fixed.)

```
// 2024-25 Q4(a) — Student / Scholarship after refactoring
// Smells fixed: (1) type code + switch on type (UG/PG) -> Replace Conditional with Polymorphism;
// (2) data clump street/postOffice/district -> Extract Class Address; (3) feature envy: Scholarship
computes CGPA and
// regularity from Student's data -> Move Method into Student; (4) duplicated code: CGPA loop in two places
-> Extract Method
// cgpa(); (5) long method / deep nesting in checkEligibility -> guard clauses; also temporary field
isFulltime only for PG
// -> moved into PostgraduateStudent; magic numbers -> named constants.
class Address {
    private final String street, postOffice, district;
    Address(String street, String postOffice, String district) {
        this.street = street; this.postOffice = postOffice; this.district = district;
    }
    public String toString() { return street + ", " + postOffice + ", " + district; }
}
abstract class Student {
    final String name, id;
    final int year;
    final Address address;
    private final float[] yearWiseGpa;
    Student(String name, String id, int year, Address address, float[] yearWiseGpa) {
        this.name = name; this.id = id; this.year = year; this.address = address; this.yearWiseGpa =
yearWiseGpa;
    }
    float cgpa() {                                          // extracted once
        float sum = 0;
        for (float g : yearWiseGpa) sum += g;
        return yearWiseGpa.length == 0 ? 0 : sum / yearWiseGpa.length;
    }
    boolean isRegular() { return yearWiseGpa.length == year; }    // moved from Scholarship
    abstract float calculateFee();
    void printDetails() {
        System.out.println("Name: " + name);
        System.out.println("Student ID: " + id);
        System.out.println("Address: " + address);
        printSpecificDetails();
    }
    protected abstract void printSpecificDetails();
}
class UndergraduateStudent extends Student {
    private static final float UG_FEE = 50000;
    UndergraduateStudent(String n, String id, int y, Address a, float[] g) { super(n, id, y, a, g); }
    float calculateFee() { return UG_FEE; }
    protected void printSpecificDetails() { System.out.println("Undergraduate, year " + year); }
}
```

```
class PostgraduateStudent extends Student {
    private static final float HIGH_CGPA = 3.5f, PG_FEE_HIGH_CGPA = 40000, PG_FEE = 48000;
    private final boolean fulltime;
    PostgraduateStudent(String n, String id, int y, Address a, float[] g, boolean fulltime) {
        super(n, id, y, a, g); this.fulltime = fulltime;
    }
    float calculateFee() { return cgpa() > HIGH_CGPA ? PG_FEE_HIGH_CGPA : PG_FEE; }
    protected void printSpecificDetails() { System.out.println("Postgraduate, " + (fulltime ? "full-time" :
"part-time")); }
}
class Scholarship {
    private static final int MIN_YEAR = 2;
    final String name; final int amount; final float cgpaThreshold;
    Scholarship(String name, int amount, float cgpaThreshold) {
        this.name = name; this.amount = amount; this.cgpaThreshold = cgpaThreshold;
    }
    boolean isEligible(Student s) {                       // guard clauses instead of nested ifs
        if (!(s instanceof UndergraduateStudent)) return false;
        if (!s.isRegular()) return false;
        if (s.year < MIN_YEAR) return false;
        return s.cgpa() >= cgpaThreshold;
    }
    void checkEligibility(Student s) {
        System.out.println(s.name + (isEligible(s) ? " is awarded scholarship: " + amount : " is not
eligible."));
    }
}
public class StudentRefactored {
    public static void main(String[] args) {
        Address a = new Address("Road 3", "Palashi", "Dhaka");
        Student ug = new UndergraduateStudent("Asef", "2205001", 3, a, new float[]{3.6f, 3.8f, 3.7f});
        Student pg = new PostgraduateStudent("Nadia", "0423001", 1, a, new float[]{3.9f}, true);
        ug.printDetails(); System.out.println("Fee: " + ug.calculateFee());
        pg.printDetails(); System.out.println("Fee: " + pg.calculateFee());
        Scholarship s = new Scholarship("Dean's", 20000, 3.5f);
        s.checkEligibility(ug); s.checkEligibility(pg);
    }
}
```

Quick Notes — Refactoring

Revision points for Lessons 18–20.

L18 · Basics

- Behaviour unchanged, structure improved; small safe steps + tests after each.
- Why: design, understanding, bugs, speed. When: rule of three, adding feature, fixing bug, review.
- Problems: databases, published interfaces. Tech debt = cost of quick fixes.

L19 · Smells → cure

- Duplicated → Extract Method · Long method → Extract Method · Long params / data clump → Parameter Object / Extract Class
- Switch / type code → Polymorphism · Feature envy → Move Method · Shotgun surgery → Move into one class
- Divergent change → Extract Class · Refused bequest → Delegation · Lazy class → Inline · Comments → Extract Method

L20 · Catalogue

- Extract Method/Class/Variable, Replace Temp with Query, Move Method, Replace Conditional with Polymorphism,
- Separate Query from Modifier, Introduce Parameter Object, Guard Clauses, lookup table for long if-chains.